

GraphQL vs REST: Um Experimento Controlado

Laboratório de Experimentação de Software

João Vitor Romero Sales

Lucas Randazzo

Novembro de 2025

Resumo

Este relatório apresenta os resultados de um experimento controlado que compara o desempenho de APIs REST e GraphQL no contexto da API do GitHub. O experimento avalia duas métricas principais: latência das respostas e tamanho do payload. Os resultados indicam que GraphQL apresenta latências 8–17% menores (significativo para cargas leves, $p < 0.05$) e payloads $\sim 37x$ menores (148 vs 5501 bytes/repo), confirmando superioridade em ambas as RQs.

Conteúdo

1	Introdução	3
1.1	Motivação	3
1.2	Objetivos	3
1.3	Hipóteses	3
1.3.1	RQ1 - Latência	3
1.3.2	RQ2 - Tamanho do Payload	3
2	Metodologia	3
2.1	Desenho Experimental	3
2.2	Variáveis	4
2.2.1	Variáveis Dependentes	4
2.2.2	Variáveis Independentes	4
2.3	Objetos Experimentais	4
2.3.1	Endpoints REST	4
2.3.2	Query GraphQL	4
2.4	Ambiente Experimental	5
3	Resultados	5
3.1	Dados Coletados	5
3.2	RQ1: Análise de Latência	5
3.2.1	Estatísticas Descritivas	5
3.3	RQ2: Análise de Tamanho do Payload	6

4	Discussão	6
4.1	Interpretação dos Resultados	6
4.1.1	RQ1 - Latência	6
4.1.2	RQ2 - Tamanho do Payload	6
4.2	Ameaças à Validade	6
4.2.1	Validade Interna	6
4.2.2	Validade Externa	6
4.2.3	Validade de Construção	6
5	Conclusão	6
5.1	Trabalhos Futuros	7
A	Código-fonte	7

1 Introdução

A linguagem de consulta GraphQL, proposta pelo Facebook em 2015, representa uma alternativa moderna às populares APIs REST. Baseada em grafos, a linguagem permite que usuários consultem bancos de dados através de schemas tipados, possibilitando que clientes especifiquem exatamente quais dados necessitam em uma única requisição.

Por outro lado, APIs construídas com a abordagem REST baseiam-se em endpoints pré-definidos, onde cada operação retorna um conjunto fixo de dados. Esta característica pode resultar em dois problemas conhecidos: *over-fetching* (receber mais dados do que o necessário) e *under-fetching* (necessitar de múltiplas requisições para obter todos os dados desejados).

1.1 Motivação

Desde o surgimento do GraphQL, diversas empresas migraram suas APIs REST para esta nova abordagem, incluindo GitHub, Shopify e Twitter. Entretanto, a literatura carece de evidências empíricas quantitativas sobre os reais benefícios desta migração.

1.2 Objetivos

Este experimento visa responder às seguintes questões de pesquisa:

- **RQ1:** Respostas às consultas GraphQL são mais rápidas que respostas às consultas REST?
- **RQ2:** Respostas às consultas GraphQL têm tamanho menor que respostas às consultas REST?

1.3 Hipóteses

1.3.1 RQ1 - Latência

- H_{01} : Não há diferença estatisticamente significativa entre a latência média das respostas REST e GraphQL ($\mu_{REST} = \mu_{GraphQL}$).
- H_{11} : A latência média GraphQL é menor ($\mu_{REST} > \mu_{GraphQL}$).

1.3.2 RQ2 - Tamanho do Payload

- H_{02} : Não há diferença estatisticamente significativa entre o tamanho médio das respostas ($\mu_{payload_{REST}} = \mu_{payload_{GraphQL}}$).
- H_{12} : O payload GraphQL é menor ($\mu_{payload_{REST}} > \mu_{payload_{GraphQL}}$).

2 Metodologia

2.1 Desenho Experimental

O experimento segue um delineamento fatorial completo $2 \times 3 \times 4$, com 24 tratamentos:

- **Tipo de API:** REST, GraphQL (2 níveis)

- **Tamanho do Dataset:** 100, 300, 1000 repositórios (3 níveis)
- **Período do Dia:** Madrugada, Manhã, Tarde, Noite (4 replicações)

2.2 Variáveis

2.2.1 Variáveis Dependentes

Tabela 1: Variáveis Dependentes do Experimento

Variável	Unidade	Descrição
latency_avg_s	segundos	Latência média por requisição
payload_avg_per_repo_bytes	bytes	Payload médio por repositório
requests_count	inteiro	Número de requisições HTTP

2.2.2 Variáveis Independentes

Tabela 2: Variáveis Independentes do Experimento

Variável	Níveis	Descrição
Tipo de API	REST, GraphQL	Tecnologia avaliada
Tamanho do Dataset	100, 300, 1000	Repositórios consultados
Período	4 horários	Controle temporal

2.3 Objetos Experimentais

O experimento utiliza a API do GitHub, consultando repositórios públicos Java ordenados por número de estrelas em ordem decrescente.

2.3.1 Endpoints REST

```
GET https://api.github.com/search/repositories
?q=language:Java+stars:>0
&sort=stars&order=desc
&per_page=100&page={n}
```

2.3.2 Query GraphQL

```
query ($queryString: String!, $first: Int!, $after: String) {
  search(type: REPOSITORY, query: $queryString,
    first: $first, after: $after) {
    pageInfo { hasNextPage endCursor }
    nodes {
      ... on Repository {
        nameWithOwner
        stargazerCount
        primaryLanguage { name }
      }
    }
  }
}
```

```

    url
  }
}
}
}

```

2.4 Ambiente Experimental

Tabela 3: Configuração do Ambiente

Componente	Especificação
Localização	Belo Horizonte, MG, Brasil (-03)
Data	27/11/2025
Linguagem	Python 3.12
Autenticação	Token GitHub pessoal
Rate Limit	REST: 25, GraphQL: 4980

3 Resultados

3.1 Dados Coletados

Foram realizadas 4 rodadas de coleta (replicações temporais), gerando os seguintes arquivos:

- comparativo_github_rest_graphql_madrugada.xlsx
- comparativo_github_rest_graphql_manha.xlsx
- comparativo_github_rest_graphql_tarde.xlsx
- comparativo_github_rest_graphql_noite.xlsx

Cada arquivo contém 6 observações (3 tamanhos $\times$ 2 APIs), totalizando 24 tratamentos.

3.2 RQ1: Análise de Latência

3.2.1 Estatísticas Descritivas

Tabela 4: Latência Média por Requisição (segundos) - 4 replicações

Dataset	REST	GraphQL	Diferença	p-value
100 repos	3.821	3.230	-15.5%	0.022
300 repos	3.970	3.293	-17.1%	0.014
1000 repos	3.857	3.528	-8.5%	0.156
Média Geral	3.883	3.350	-13.7%	

Teste t Student ($\alpha=0.05$): H_0 rejeitada para cargas 100/300.

3.3 RQ2: Análise de Tamanho do Payload

Tabela 5: Payload Médio por Repositório (bytes) - 24 observações

API	Média	Desvio Padrão	Razão
REST	5501	21.7	37.1x
GraphQL	148	1.4	

H0 rejeitada ($p < 0.001$): GraphQL $\sim 37x$ menor.

4 Discussão

4.1 Interpretação dos Resultados

4.1.1 RQ1 - Latência

GraphQL 13.7% mais rápido globalmente, significativo em cargas leves. Para 1000 repos ($p=0.156$), efeito ainda favorável mas não estatisticamente significativo.

4.1.2 RQ2 - Tamanho do Payload

Redução drástica (97.3%) confirma eliminação de over-fetching: GraphQL retorna apenas 4 campos vs todos do REST.

4.2 Ameaças à Validade

4.2.1 Validade Interna

- **Rate limiting:** Mitigado por uso de token autenticado e delays.
- **Variabilidade de rede:** Controlada por replicações em diferentes horários.

4.2.2 Validade Externa

- **Generalização limitada:** Resultados específicos para API do GitHub.
- **Tipo de consulta:** Apenas busca de repositórios foi testada.

4.2.3 Validade de Construção

- **Campos diferentes:** GraphQL solicita subconjunto; REST retorna todos.
- **Medição de tempo:** `time.perf_counter()` fornece precisão adequada.

5 Conclusão

1. **RQ1:** Sim, GraphQL mais rápido (8-17%, significativo em cargas ≤ 300 repos).
2. **RQ2:** Sim, GraphQL $\sim 37x$ menos payload (5501 vs 148 bytes/repo).

H0 rejeitada em ambas RQs. GraphQL superior para eficiência de rede e latência em cenários reais.

5.1 Trabalhos Futuros

Testes com mutations, outras APIs, carga servidor, aplicações móveis.

Referências

1. GraphQL Specification. <https://graphql.org/>
2. GitHub GraphQL API. <https://docs.github.com/en/graphql>
3. GitHub REST API. <https://docs.github.com/en/rest>
4. Wohlin, C. Experimentation in Software Engineering. Springer, 2012.

A Código-fonte

O código-fonte completo do experimento está disponível em:

<https://github.com/o-romeroo/lab-experimentacao-05>